will return a maximum of 5 clients and because it specifies no offset it will return the first 5 in the table. The SQL it executes looks like this:

```
SELECT * FROM clients LIMIT 5
```

Adding `offset` to that

```
Client.limit(5).offset(30)
```

will return instead a maximum of 5 clients beginning with the 31st. The SQL looks like:

```
SELECT * FROM clients LIMIT 5 OFFSET 30
```

6 Group

To apply a `GROUP BY` clause to the SQL fired by the finder, you can specify the `group` method on the find. For example, if you want to find a collection of the dates orders were created on:

```
Order.select("date(created_at) as ordered_date, sum(price) as
total_price").group("date(created_at)")
```

And this will give you a single `Order` object for each date where there are orders in the database.
The SQL that would be executed would be something like this:

```
SELECT date(created_at) as ordered_date, sum(price) as total_price
FROM orders
GROUP BY date(created_at)
```

6.1 Total of grouped items

To get the total of grouped items on a single query call `count` after the `group`.

```
Order.group(:status).count
# => { 'awaiting_approval' => 7, 'paid' => 12 }
```

The SQL that would be executed would be something like this:

```
SELECT COUNT (*) AS count_all, status AS status
FROM "orders"
GROUP BY status
```

7 Having

SQL uses the `HAVING` clause to specify conditions on the `GROUP BY` fields. You can add the `HAVING`clause to the SQL fired by the `Model.find` by adding the `:having` option to the find.
For example:

```
Order.select("date(created_at) as ordered_date, sum(price) as
total_price").
  group("date(created_at)").having("sum(price) > ?", 100)
```

The SQL that would be executed would be something like this:

```
SELECT date(created_at) as ordered_date, sum(price) as total_price
FROM orders
GROUP BY date(created_at)
HAVING sum(price) > 100
```

This will return single order objects for each day, but only those that are ordered more than $100 in a day.